CRESTRON SIMPL#PRO PROGRAM LIBRARY

CwsDebug

Self-contained CWS debug log page for Crestron SIMPL#Pro programs

Author	Mark Jervis
Company	Stage Front Presentation Systems
Version	1.6.0
Date	2026-10-14
Assembly	CwsDebug.dll (Crestron.SimplSharp.SDK.ProgramLibrary 2.21.257, net8.0) References Crestron.SimplSharpPro.DeviceSupport
Namespace	CwsDebug
Public types	CwsDebugPage; CwsErrorLog and CwsConsole (drop-in replacements for Crestron's ErrorLog and CrestronConsole)

Overview

CwsDebug hosts a small, self-contained web page on the Crestron Web Server (CWS) that streams your program's own log messages and — on a hardware appliance — the processor error log. The page provides live search, severity filtering, light/dark themes, a pause control, a master logging switch, and one-click text export. It also hosts a **Signals panel** for viewing and driving a registered device's signals live, in either direction. The entire UI (markup, CSS, and JavaScript) is embedded in the library, so there are no external content files to deploy.

The log is held in a bounded in-memory ring buffer (size configurable at registration) and is **mirrored to disk**, so the most recent entries survive a program restart. Messages are stamped to millisecond resolution, and the page fetches only new entries on each poll. A process-wide **singleton** exposes the whole API statically, so any class on any thread can log without being handed an instance.

> **API note (since 1.1.0):** the API is *static*. Call `CwsDebugPage.LogInfo(…)` and `CwsDebugPage.Register(…)` directly — do *not* construct a `new CwsDebugPage()` (the constructor is private).

It is built as a **Program Library** and referenced by a SIMPL#Pro program; at build time the program bundles `CwsDebug.dll` into its own program archive (`.cpz`). The library is not separately loadable onto a processor.

Example

The debug page running live on an appliance. The header reads `TEST - Debugger` (the host program is named *TEST*); each entry shows its source, severity badge, message, and timestamp, with search and level filters above.



CwsDebug page at /cws/debugger/debugger.html (dark theme). Later builds add Logging and Pause controls to the header.

Installation

Add a reference to the library from your SIMPL#Pro program project:

```
<ItemGroup>
  <ProjectReference Include="..\CwsDebug\CwsDebug\CwsDebug.csproj" />
</ItemGroup>
```

(Or reference the compiled `CwsDebug.dll` directly.)

Usage

The API is static, so there is nothing to instantiate. Call `Register()` once from `InitializeSystem()`; you can log from anywhere, on any thread, even before `Register()` runs (early entries are kept and shown once the page is up).

```
using CwsDebug;

public class ControlSystem : CrestronControlSystem
{
    public override void InitializeSystem()
    {
        CwsDebugPage.Register("/debugger");          // default base path; buffer
defaults to 2000 entries
        // CwsDebugPage.Register("/debugger", 5000);  // optional: larger ring buffer
        // CwsDebugPage.MinLevel = CwsDebugPage.DebugLevel.Notice;  // optional: drop
anything below Notice
```

```
        CwsDebugPage.LogInfo("System starting");
        CwsDebugPage.LogNotice("Loaded {0} devices", deviceCount);
        CwsDebugPage.LogWarning("Low buffer on {0}", deviceName);
        CwsDebugPage.LogError("Comms timeout on {0}", deviceName);
        CwsDebugPage.LogFatal("Unrecoverable: {0}", reason);   // also written to the
system ErrorLog
    }

    public override void InitializeSystem_Example_Catch()
    {
        try { /* ... */ }
        catch (Exception e)
        {
            CwsDebugPage.LogException(e, "Error during init");  // full type + message +
stack trace
        }
    }
}
```

Cleanup is automatic — the library releases the CWS server and flushes the log to disk when the program stops or restarts — so calling `Dispose()` / `Shutdown()` by hand is optional.

Browsing to the page

Platform	URL
Appliance (RMC4, etc.)	`http://<processor>/cws/debugger/debugger.html`
VC-4 (Server)	`http://<processor>/VirtualControl/Rooms/<roomId>/cws/debugger.html`

API reference

All members are `static` on `CwsDebugPage`.

Member	Description
`LogInfo(msg, args)`	Records an informational message.
`LogNotice(msg, args)`	Records a notice-level message.
`LogWarning(msg, args)`	Records a warning-level message. *(new in 1.2.0)*
`LogError(msg, args)`	Records an error-level message.
`LogFatal(msg, args)`	Records a fatal message and mirrors it to the system error log.

`LogException(ex, context?)`	Records an exception at Error level with its full type, message and stack trace (and inner exceptions). *(new in 1.2.0)*
`Register(basePath, maxEntries?)`	Starts the CWS server. `basePath` defaults to `"/debugger"`; `maxEntries` defaults to `2000` (values ≤ 0 are ignored).
`MinLevel` PROP	Minimum severity recorded (`DebugLevel` = Info, Notice, Warning, Error, Fatal). Below-floor messages are dropped at the source. Defaults to `Info`. *(new in 1.2.0)*
`LoggingEnabled` PROP	Master record switch. When `false`, all `Log*` calls are no-ops (fatals are still mirrored to the system ErrorLog). Persisted across restarts. Defaults to `true`. *(new in 1.2.0)*
`Dispose()` / `Shutdown()`	Stops the server, flushes the log, releases resources. Idempotent and optional (cleanup is automatic).
`RegisterDevice(name, device, sgdPath?)`	Exposes a device's signals in the Signals panel. Accepts any `BasicTriList` (XPanel, TSW, and other touchpanels). Safe from any thread and before `Register()`. Pass the same SGD path given to `LoadSmartObjects` to have smart objects labelled with their design-time names. *(new in 1.3.0; `sgdPath` in 1.5.0)*
`RegisterContract(name, device, cse2jPath)`	As `RegisterDevice`, and additionally loads a Crestron Construct contract so signals can be addressed by their contract item path. If the manifest is missing or unreadable the device still registers and numeric addressing still works, with a warning in the log. *(new in 1.5.0)*
`UnregisterDevice(name)`	Drops a registered device; call before disposing it. Returns `false` if the name was not registered. *(new in 1.3.0)*
`UnregisterAllDevices()`	Drops every registered device. *(new in 1.3.0)*
`ValueWritesAllowed` PROP	Master switch for changing signals, in either direction. When `false` the Signals panel stays read-only whatever the page does. Defaults to `true`. Not persisted. *(new in 1.3.0)*
`ValueWritesArmed` PROP	Arm gate for changing signals. Defaults to `false` at every program start and is never persisted. *(new in 1.3.0)*
`ValueArmTimeoutMs` PROP	Idle time before the arm gate closes itself. Defaults to `300000` (5 minutes); `≤ 0` disables the timeout. *(new in 1.3.0)*

Each `Log*` method accepts an optional `string.Format` template plus arguments, e.g. `CwsDebugPage.LogInfo("Connected to {0}:{1}", host, port);`

Two further static classes, `CwsErrorLog` and `CwsConsole`, stand in for Crestron's `ErrorLog` and `CrestronConsole` — see *Migrating from ErrorLog / CrestronConsole* below. *(new in 1.6.0)*

Member	Description
`CwsErrorLog.Error / Info / Notice / Ok / Warn(msg, args)`	Same overloads as `ErrorLog`; records on the page at the mapped level.
`CwsErrorLog.Exception(msg, ex)`	Records `msg` and the full exception text at Error level.
`CwsErrorLog.MirrorToErrorLog` PROP	Also forward every call to the real `ErrorLog`. Defaults to `false`.
`CwsConsole.Print / PrintLine(msg, args)`	Same overloads as `CrestronConsole`; line-buffered onto the page.
`CwsConsole.Flush()`	Publishes any pending `Print` fragment (text with no newline yet) to the page immediately.
`CwsConsole.PrintLevel` PROP	Level console output is recorded at (`DebugLevel`). Defaults to `Info`. At `Fatal`, lines behave like `LogFatal`: recorded as Fatal and also written to the processor error log.
`CwsConsole.PassThrough` PROP	Also forward every `Print` / `PrintLine` call to the real `CrestronConsole`. Defaults to `false`.
`CwsConsole.AddNewConsoleCommand` (2 overloads), `RemoveConsoleCommand`, `ConsoleCommandResponse` (2 overloads), `SendControlSystemCommand` (4 overloads: `ref string pResponse` ×2, `out StringBuilder? pResponse` ×2), `ConsoleRegistered` PROP	The rest of `CrestronConsole`'s public surface, forwarded unchanged to the real `CrestronConsole`: same signatures, return values and exceptions. Not gated by `PassThrough`, nothing recorded on the page. `ConsoleCommandResponse` is always forwarded — it is the reply to whoever typed the console command. `ConsoleRegistered` is get-only.

Migrating from ErrorLog / CrestronConsole

Code that already logs through Crestron's `ErrorLog` and `CrestronConsole` can write to the debug page without touching a single call: alias the two names to CwsDebug's drop-in classes at the top of each file. Both classes cover their SDK class's entire public static surface, with overloads matching the SDK's own (same names, parameter names and types, `ref` / `out`), so every existing call binds unchanged and aliasing compiles even where the file also registers console commands. The logging calls (`ErrorLog.*`, `Print`, `PrintLine`) never throw; `CwsConsole`'s other members (`AddNewConsoleCommand`, `RemoveConsoleCommand`, `ConsoleCommandResponse`, `SendControlSystemCommand`, `ConsoleRegistered`) are pure pass-throughs to the real `CrestronConsole`, with the same return values and exceptions.

```
using ErrorLog        = CwsDebug.CwsErrorLog;
using CrestronConsole = CwsDebug.CwsConsole;
```

Before — the calls go to the processor error log and the console:

```
using Crestron.SimplSharp;

ErrorLog.Notice("Loaded {0} devices", deviceCount);
ErrorLog.Exception("Error during init", e);
CrestronConsole.PrintLine("Projector power: {0}", state);
```

After — the same calls, now recorded on the debug page:

```
using Crestron.SimplSharp;                    // still needed for everything else
using ErrorLog        = CwsDebug.CwsErrorLog;
using CrestronConsole = CwsDebug.CwsConsole;

ErrorLog.Notice("Loaded {0} devices", deviceCount);
ErrorLog.Exception("Error during init", e);
CrestronConsole.PrintLine("Projector power: {0}", state);
```

How each call lands on the page

Crestron call	Page level
`ErrorLog.Info`	Info
`ErrorLog.Notice`	Notice
`ErrorLog.Ok`	Notice (the page has no OK level)
`ErrorLog.Warn`	Warning
`ErrorLog.Error`	Error
`ErrorLog.Exception(msg, ex)`	Error — `msg`, a new line, then the full exception text
`CrestronConsole.Print` / `PrintLine`	`CwsConsole.PrintLevel` (Info unless changed)

Settings

Setting	Default	Effect
`CwsErrorLog.MirrorToErrorLog`	`false`	Also forward every `ErrorLog` call, with its original arguments, to the real `ErrorLog`.

`CwsConsole.PassThrough`	`false`	Also forward every `Print` / `PrintLine` call, with its original arguments, to the real console.
`CwsConsole.PrintLevel`	Info	The level all console output is recorded at on the page. At `Fatal`, each line behaves like `CwsDebugPage.LogFatal`: recorded as Fatal on the page and also written to the processor error log.

Forwarding happens regardless of `LoggingEnabled` and `MinLevel`; recording on the page honours both, exactly as the `Log*` methods do.

What is different

- **By default these calls write ONLY to the debug page** — nothing reaches the processor error log or the console. Turn on `MirrorToErrorLog` / `PassThrough` to keep the old destinations too.
- **Mirroring shows each message twice on an appliance:** once in the app log on the page, and again in the processor error log view.
- `Print` **is line-buffered.** The page is a list of entries, not a character stream, so `Print` text is held until a newline arrives; each complete line becomes one entry, and `PrintLine` flushes whatever is pending. Lines reach the page in the order they were buffered, even across threads. A fragment with no newline is flushed once it passes 4096 characters. A fragment left without a newline is published as its own entry once it has been pending at least 2 s, on the library's periodic 5-second tick — so typically 2–7 s later, and only once `CwsDebugPage.Register()` has run — so `Print("Connecting...")` still shows up, timestamped when it is published rather than when `Print` was called. Call `CwsConsole.Flush()` to publish a pending fragment immediately; any pending fragment is also flushed at program stop. As a result, text printed in pieces with long pauses between them can appear split across entries.
- **Blank lines are skipped.**
- `Ok` **is recorded as Notice.**
- **There is no Fatal equivalent** — `ErrorLog` has none to alias. Keep using `CwsDebugPage.LogFatal` for fatal messages.
- `Error(msginfo, errorstring)` follows the SDK: `msginfo` is the method name or other context, `errorstring` the error text, and the page shows `msginfo: errorstring`. As a convenience, if `msginfo` uses argument 0 as a format placeholder — in any form .NET accepts, such as `{0}`, `{0:X}` or `{0,8}`, but not the escaped `{{0}}` — it is formatted with `errorstring` instead. That covers `Error("Failed {0}", x)` with a string `x`, which C# binds to this overload rather than the template-plus-args one.
- **Console commands,** `ConsoleCommandResponse` **and** `SendControlSystemCommand` **are unchanged.** They forward straight to the real `CrestronConsole` whatever `PassThrough` is set to, and nothing they do is recorded on the page.

Signals panel

Registering a device exposes its signals on the page, where they can be viewed live and driven — the same workflow as Crestron Toolbox, without leaving the browser. One call is all the program needs:

```
public override void InitializeSystem()
{
    CwsDebugPage.Register("/debugger");
    CwsDebugPage.RegisterDevice("Panel", tsPanel);   // any BasicTriList
}
```

Call `UnregisterDevice` before disposing the device. A program's own stop handler usually runs before the library's, so without it the page can briefly reach a disposed panel:

```
CwsDebugPage.UnregisterDevice("Panel");
panel.Dispose();
```

Adding a signal

Open **Signals** in the header and choose a device, a type (Digital, Analog or Serial) and a direction:

- **In** — signals going *into* the panel: button feedback, gauge levels, indirect text. Driving these changes what the panel *shows*.
- **Out** — signals coming *out of* the panel: presses and slider moves. Driving these makes the program react exactly as if the panel had been used, so they change what the system *does*.
- **Both** — one row showing a signal's two directions together.

> These names follow the panel's point of view, matching Crestron's own naming: *in* to the panel, *out* of the panel. They are the reverse of 1.4.0 and earlier, so a saved row may now carry the opposite label — the signal it refers to has not changed.

Entering a **Smart Object ID** lists that object's signals by name (`Item 1 Pressed`, `Select Item`, `Item 1 Text`) instead of by number, so there is no need to work out which sig number a control uses. The field offers the objects it knows about, so a number or part of a name is enough to find one. Rows can be given friendly names, and the list is remembered in the browser rather than declared in the program.

Smart objects are labelled with their own names when the program supplies them: from the SGD for a Smart Graphics panel (`Button List`, `Simple Keypad`), or from the contract for a Crestron Construct panel (`Main.Dpad`, `Main.HButtonList`), whose signals are then listed by contract name (`Up`, `Down`, `Left` …) rather than as generic numbered signals.

```
tsPanel.LoadSmartObjects(sgdPath);
CwsDebugPage.RegisterDevice("Panel", tsPanel, sgdPath);          // Smart Graphics
CwsDebugPage.RegisterContract("Touchpanel", ch5Panel, "My Project.cse2j");   // Construct
/ CH5
```

A Construct manifest is the `<Project>.cse2j` file from the project's `output\contract\mapping\` folder; the program must ship it as content so it exists on the processor. With it loaded, a signal can also be addressed by its contract item path.

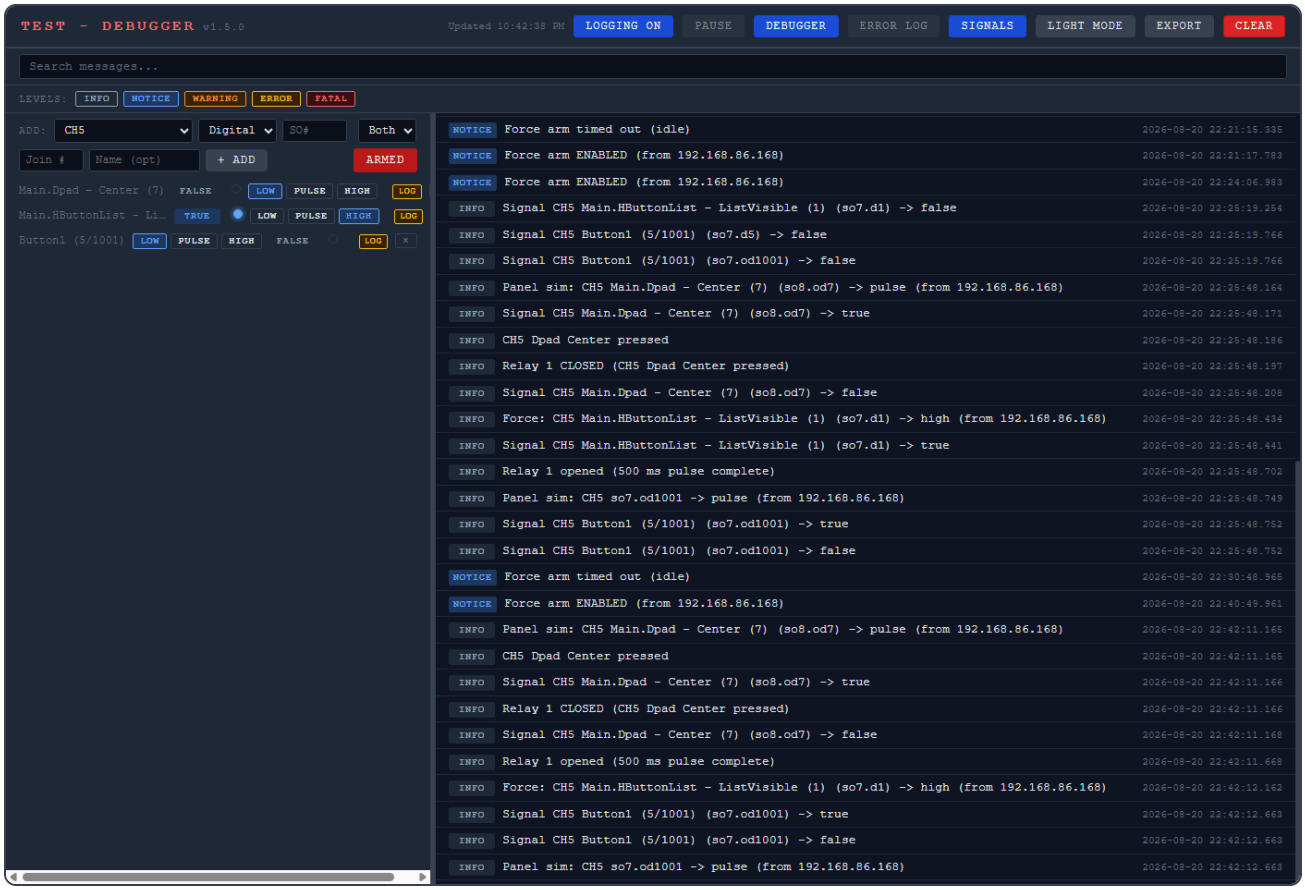
For a Construct panel, the signal picker only offers joins the manifest actually names. A smart object can expose more raw joins than the contract declares (a Dpad's directions, for instance, are events with no matching state) — those are still reachable by typing a join number directly, just not listed by a name that does not exist.

Viewing and driving

Digital rows show the signal's state and light a lamp for as long as it is high — a press too brief to fall on a poll still registers as a flash. Analog and serial rows are edited by typing a value and pressing Enter; a value that changes on the system while a field is being edited replaces what was typed, so a force can never overwrite a change it has not shown.

Digital signals are driven with **Low**, **Pulse** and **High**, in both directions. A value is written to the signal itself: unlike Toolbox there is no separate "unforce", so it stands until the program or the panel changes it again.

The two directions do different things. Driving an **In** signal changes what the panel *shows* — a button lights, a gauge moves — and nothing else. Driving an **Out** signal makes the program react as though the panel had been used: its own signal handlers, and Construct contract events, run normally, so the code behind a control can be exercised from the page without touching a panel and without adding anything to the program. Where that is not available on a device the controls are not shown, and the row stays a live read-out.



*The Signals panel beside the log, with digital, analog, serial and Smart Graphics signals. Two rows have **Log** switched on, and the log shows the resulting entries for both directions of a button. The divider between the two panes can be dragged.*

Logging a signal's changes

Each row also carries a **Log** toggle. With it on, the processor writes a line to the log every time that signal changes:

```
Signal Panel Power Button (od1) -> true
Signal Panel Volume (a1) -> 46536
```

That gives a running trace of any join without adding a single `Log*` call to the program — useful for watching what a panel is actually sending, or confirming what the program drives back. The lines are written on the processor rather than in the browser, so they carry the moment the change happened and include transitions too brief to land on a poll.

- Digital edges are always logged. Analog and serial have a floor between entries, so a dragged slider cannot flood the buffer.
- Entries go through the normal log path, so the master logging switch and the severity floor still apply — unlike a force, which is always recorded.

- Logging a signal stops shortly after the page stops asking for it, so closing the browser cannot leave a processor logging every transition indefinitely.

Keeping changes safe

- Changing a signal is off until it is armed on the page. Arming is never persisted: it is forgotten whenever the program restarts or the page is reloaded, and lapses after `ValueArmTimeoutMs` (default five minutes) without activity.
- `ValueWritesAllowed = false` makes the panel read-only regardless of the page — the intended setting for a system being handed over, since it keeps the live view.
- Every change and every refusal is written to the log with the address it came from, ignoring both the master logging switch and the severity floor, so the record cannot be turned off.
- Both directions share one arm gate, one kill switch and one audit trail; driving the Out direction runs program logic, so it is never on a lighter footing than an In change.
- Values are never persisted.

What stops the page updating

Three things are easily confused:

Control	What it stops
Logging	Recording on the processor. It does *not* stop the page requesting updates.
Pause	The page requesting updates. The processor keeps recording; resuming catches up.
(automatic)	Requests, while the tab is hidden and after fifteen idle minutes.

Signals refresh about four times a second while the panel is open, so a button press appears in roughly a quarter of a second; the log refreshes every two seconds. Only one request is ever in flight, so a slow connection simply refreshes less often instead of queueing requests. The automatic stop counts arriving values as activity, so watching a panel being pressed keeps the page live even with no mouse or keyboard use; it resumes on a click, a keystroke, or the tab being shown.

Configuration & persistence

- **Buffer size:** the log is a bounded ring buffer; once full, the oldest line is dropped as each new one arrives. Size it per program with `Register("/debugger", maxEntries)` (default 2000). This bounds both memory and the JSON payload sent to the page.
- **Severity floor (`MinLevel`):** drops sub-threshold messages before they are ever stored — distinct from the page's level-filter buttons, which only hide already-recorded entries in the browser.
- **Master switch (`LoggingEnabled` / "Logging" button):** turns recording on for a debugging session and off when finished. The state is persisted, so it survives a restart. When off, the page

status shows "LOGGING OFF".

- **Persistence:** the buffer is mirrored to the application directory (`cwsdebug_<path>.log` , plus `.state` for the switch) every few seconds when changed and once on program stop, then reloaded on the next `Register()` — so recent history survives a restart.
- **Pause:** a page-side control that freezes polling/rendering. Recording continues on the processor, so resuming shows everything logged during the freeze (subject to the ring cap).

Platform behavior

- **Base path (VC-4):** with the default base path, the redundant segment is collapsed so the page is served directly at `.../cws/debugger.html` . Passing an explicit (non-default) base path keeps it in the URL on every platform — required if a single program hosts more than one debug page.
- **Concurrent rooms (VC-4):** each room is an isolated program instance with its own CWS namespace and its own log, so multiple rooms run side-by-side without conflict.
- **Header title:** shown as `"{Name} - Debugger"` , where Name is the *room name* on VC-4 and the *host program's namespace/assembly name* on an appliance, falling back to the controller prompt name, then the base-path segment, then "Debugger".
- **Processor error log:** read via the console `err` command and shown only on a hardware appliance; the feature is hidden on VC-4, where no per-program console error log exists.

A note on network access

The debug page has no login. Anyone who can reach the processor on the network can open the page, read the log, clear it, or switch logging off — the same way Crestron's own built-in web pages work. On a dedicated control-system network this is normally fine. If the processor sits on a network shared with general office traffic or guests, keep in mind that the log is visible to everyone on that network — so avoid logging passwords, door codes, or anything else sensitive, and ask your IT/network administrator to place the processor on a restricted VLAN if that's a concern.

The same applies to the Signals panel, which can change values on a running system. Forcing is switched off until someone arms it, arming is forgotten whenever the program restarts or the page is reloaded, it switches itself back off after five idle minutes, and every force is written to the log with the address of the machine that sent it. For a system being handed over, set `CwsDebugPage.ValueWritesAllowed = false` to leave the panel read-only while keeping the live view.

Changelog

Full history; the same content as the repository's `CHANGELOG.md`.

1.6.0 — 2026-10-14

Added

- **Drop-in replacements for `ErrorLog` and `CrestronConsole`** (suggested in #7). Existing code that logs through Crestron's static classes can write to the debug page by changing one using-alias per class, with no edits at the call sites: `using ErrorLog = CwsDebug.CwsErrorLog;` and `using CrestronConsole = CwsDebug.CwsConsole;`. Both classes cover their SDK class's entire public static surface with the SDK's own signatures, so existing calls bind unchanged and aliasing compiles even where the file also registers console commands. The logging methods (`ErrorLog.*`, `Print`, `PrintLine`) never throw.
- **`CwsErrorLog`** mirrors `ErrorLog`: `Error` (template plus args, a single string, and the `(string, string)` pair), `Exception(string, Exception)`, `Info`, `Notice`, `Ok` and `Warn`. Levels map as Info to Info, Notice to Notice, Warn to Warning, Error and Exception to Error, and Ok to Notice (the page has no OK level). `Exception` records the message followed by the full exception text — type, message, stack trace and inner exceptions. The `params` overloads accept nullable arguments (`object?[]`) exactly like `ErrorLog`'s, so nullable-enabled callers get no new warnings.
- **`CwsConsole`** mirrors `CrestronConsole.Print` / `PrintLine`. The console is a character stream but the page is a list of entries, so output is line-buffered: `Print` text is held until a newline arrives, each complete line becomes one entry, `PrintLine` flushes whatever is pending, and blank lines are skipped. Lines reach the page in the order they were buffered, even across threads. A fragment left without a newline is published as its own entry once it has been pending at least 2 s, on the library's periodic 5-second tick — so typically 2–7 s later, and only once `CwsDebugPage.Register()` has run — timestamped when it is published rather than when `Print` was called. The new `CwsConsole.Flush()` publishes a pending fragment immediately, and any pending fragment is flushed at program stop, so text printed in pieces with long pauses between them can appear split across entries. `CwsConsole.PrintLevel` sets the level all console output is recorded at (defaults to Info); at Fatal, lines behave like `CwsDebugPage.LogFatal` — recorded as Fatal on the page and also written to the processor error log.
- **`CwsConsole`** also carries the rest of `CrestronConsole`'s public surface as pure pass-throughs to the real class — `AddNewConsoleCommand` (2 overloads), `RemoveConsoleCommand`, `ConsoleCommandResponse` (2 overloads), `SendControlSystemCommand` (4 overloads: `ref string` and, as in the SDK, `out StringBuilder` responses) and the get-only `ConsoleRegistered` — with the same signatures, return values and exceptions. They are not gated by `PassThrough` and record nothing on the page; `ConsoleCommandResponse` is always forwarded, since it is the reply to whoever typed at the console.

- `CwsErrorLog.MirrorToErrorLog` and `CwsConsole.PassThrough` (both default off) also forward every logging call, with its original arguments, to the real `ErrorLog` / `CrestronConsole` (`PassThrough` covers `Print` / `PrintLine`). Forwarding happens even when `LoggingEnabled` or `MinLevel` keeps the message off the page.

By default these classes write ONLY to the debug page — not to the processor error log and not to the console. Turn on `MirrorToErrorLog` / `PassThrough` to keep the old destinations as well; on an appliance a mirrored `ErrorLog` message then shows up twice on the page (once in the app log, once in the processor error log view). Page recording honours `LoggingEnabled` and `MinLevel` like the `Log*` methods.

`Error(msginfo, errorstring)` follows the SDK: `msginfo` is the method name or other context, `errorstring` the error text, and the page shows `msginfo: errorstring`. As a convenience, if `msginfo` uses argument 0 as a format placeholder — in any form .NET accepts, such as `{0}`, `{0:X}` or `{0,8}`, but not the escaped `{{0}}` — it is formatted with `errorstring` instead. That covers a call such as `Error("Failed {0}", x)` with a string `x`, which C# binds to this overload rather than to the template-plus-args one.

1.5.1 — 2026-08-21

Fixed

- A contract device's signal picker no longer offers joins the manifest doesn't declare a name for. Those were shown with CH5's own generic placeholder ("Bool Sig Number 7"), which read as a mismatch against the same smart object's other direction when that one has a real contract name (e.g. a Dpad's Out picker offering "Up", "Down"... while its In picker offered a meaningless placeholder for an unrelated, undeclared join). Manual Join# entry still reaches anything the picker now leaves out. SGD-loaded devices are unaffected — their SDK name is already meaningful, not a placeholder.
- The PDF documentation no longer carries a browser-print header (title/date) and footer (source file path) on every page.

1.5.0 — 2026-08-20

Added

- **Crestron Construct / CH5 contracts** — `CwsDebugPage.RegisterContract(name, device, cse2jPath)` registers a panel and loads its Construct contract, so its signals can be addressed by their contract item path (`c~Main.B1.Press`) as well as by join number. The manifest is the project's own `<Project>.cse2j`, which the host program ships as content. If it is missing or unreadable the device still registers and numeric addressing still works — a warning says so rather than failing silently.
- **Named smart objects.** Smart-object numbers are labelled with the object's own name, taken from the SGD for a Smart Graphics panel (`Button List`, `Simple Keypad`) or from the contract for

a Construct panel (`Main.Dpad`, `Main.HButtonList`). `RegisterDevice` takes an optional `sgdPath` to supply the SGD; without it, numbers are still listed, just unlabelled. The SO field offers the known objects as a searchable list — type a number or part of a name.

- **Named signals inside a smart object.** Contract-backed objects list their signals by contract name (`Left`, `Button3.ButtonPress`) instead of the generic `Bool Sig Number 5`.
- **Paired rows for contract items.** A contract's press and its selected-state can be shown as one row even though they sit on different joins.
- **Driving the Out direction.** The **Pulse / High / Low** controls now work on Out signals too. Driving one makes the program react exactly as if the panel had been used — its own signal handlers and Construct contract events run — so a control on the page can exercise the code behind a control on the panel, with no changes to the program. Subject to the same arm gate, kill switch and audit trail as the In direction. Where it is unavailable the controls do not appear and the row stays a live read-out.
- The **Log** toggle is on by default for a newly added signal.
- Clicking the clock in the page header switches between 12- and 24-hour time.

Changed

- **Signal directions are now named from the panel's point of view, matching Crestron's own naming: `In` goes *into* the panel (feedback, text — changes what the panel shows) and `Out` comes *out of* it (presses — changes what the system does).** This is the reverse of 1.4.0, so a saved row may now show the opposite label; the signal it refers to has not changed. The signal keys were already API-aligned (`od5` is an Out signal) and are unchanged.
- Both directions are driven the same way, with **Low**, **Pulse** and **High**.
- The direction selector comes after the smart-object field, since which directions are available depends on the object chosen.
- The Signals button appears only while at least one device is registered, and follows registration while the page is open.
- Signals are polled with a request body rather than a query string, so a long list of logged signals cannot outgrow the request.

Fixed

- A signal row is identified by the row itself, so renaming, un-logging or removing one row no longer affects another that happens to share a join.
- Adding a second valid paired row that shares one of its joins is no longer silently ignored.
- A signal that could not be resolved when it was first requested is retried, so a row added before the program finished starting recovers instead of showing an error for good.
- A contract's smart-object names are no longer left blank when the project has a page-level signal with no dotted path.

- One malformed entry in a contract manifest no longer discards the whole contract.
- Duplicate item paths in a manifest keep their first declaration and are reported, rather than silently changing a signal's direction.
- Reaching a device by a differently-cased name no longer writes duplicate log lines for the same change.
- The persisted log keeps a recoverable copy on disk throughout a flush, so power loss during a write cannot lose the history.
- Requests are bounded and guarded: a capped watch list and key length, a hard cap on cached resolutions, and a top-level guard so a browser closing mid-request cannot surface an error.
- Signal-change subscriptions are attached and detached outside the library's own lock, removing a potential stall when a busy panel is re-registered.
- Log lines and error replies are never written while the device lock is held.
- The force arm timer is read atomically, so it cannot mis-expire on a 32-bit processor.
- Reading a Smart Graphics file is bounded and streamed rather than buffered whole.

1.4.0 — 2026-08-12

Added

- **Signal logging** — each signal in the Signals panel has a **Log** toggle. With it on, the processor writes a line every time that signal changes (`Signal XPanel Power Button (od1) -> true`), so a join can be followed without adding anything to the program. Lines come from the processor, so they carry the moment the change happened and include transitions too brief to fall on a poll. Digital edges always log; analog and serial have a floor between entries so a dragged slider cannot flood the buffer. Logging stops by itself shortly after the page stops asking, and the master logging switch and severity floor still apply.

Changed

- Signals poll on their own 250 ms cadence rather than the log's two seconds, so a button press reaches the page in about a quarter of a second. One request is in flight at a time, so a slow connection throttles itself instead of queueing.
- The page stops requesting updates while its tab is hidden, and after fifteen minutes in which neither the user touched the page nor any value arrived. Arriving values count as activity, so watching a panel being pressed never suspends it. It resumes on a click, a keystroke, or the tab being shown.
- The Logging and Pause tooltips now state which one is a recording switch on the processor and which one stops the page requesting updates.

1.3.0 — 2026-08-11

Added

- **Signals panel** — a device's joins can be viewed and forced live from the debug page, alongside the log. `CwsDebugPage.RegisterDevice(name, device)` exposes any `BasicTriList` (XPanel, TSW, and other touchpanels); `UnregisterDevice` / `UnregisterAllDevices` release it.
- Digital, analog and serial joins in both directions, plus Smart Graphics object signals addressed by their SGD names. Joins that exist in both directions can be shown as a single paired row.
- Forcing: **High**, **Low** and **Pulse** on digital joins, and typed values on analog and serial joins. Signals coming *from* a panel are read-only, since output signals cannot be written.
- Force safety: `ValueWritesAllowed` disables forcing entirely for a handed-over system; forcing is otherwise off until armed on the page, arming is never persisted and lapses after `ValueArmTimeoutMs` (default 5 minutes) of inactivity; every force and every refusal is written to the log with the requesting address, regardless of the logging switch or severity floor.
- The signal list is built on the page, with friendly names per row, and is remembered in the browser rather than declared in the program.
- Panel layout: the signals panel and the log sit side by side with a draggable divider.

Changed

- The `APP` / `SYS` source tag on log rows is shown only when both log sources are displayed.
- Buttons, dropdowns and fields across the page have hover help.

Requirements

- Adds a reference to `Crestron.SimplSharpPro.DeviceSupport`.

1.2.1 — 2026-08-11

Fixed

- Log messages containing tab or other control characters no longer break the page's JSON feed (previously one such entry made the page show "Connection error" until the entry aged out of the buffer). All control characters are now escaped in API responses.
- `Log*` calls can no longer throw a `FormatException` into the calling program — a mismatched template/argument pair or literal braces now falls back to recording the raw template with the arguments appended. `LogFatal` mirrors pre-formatted text to the system error log the same way.
- Messages logged before `Register()` runs are now reliably persisted; previously they could be lost on restart if nothing else logged afterward.
- Registering a base path already owned by another program on the same appliance no longer throws out of `InitializeSystem()`; the conflict is reported to the system error log and the page is simply unavailable. A retry after a failed registration no longer duplicates the persisted log or leaks timers.

- `Register()` is marked `NoInlining` so the host program's name resolves correctly in the page header regardless of JIT inlining.
- The page title (room/program name) is HTML-encoded before being placed in the page.
- Processor error-log timestamps are normalized (invariant culture) so the combined Debugger + Error Log view sorts chronologically across both sources.
- The processor error-log pane updates correctly once the console log is at capacity, and a transient fetch failure no longer delays rendering of already-received entries.

Changed

- Ring-buffer trimming is amortized: the buffer may transiently hold up to ~10% more than `maxEntries` and is trimmed back in one operation, making logging under load significantly cheaper. At least `maxEntries` entries are always retained.
- Incremental log fetches locate the client's cursor by binary search.
- The page only rebuilds its display when a poll actually returns new data.
- Registration is thread-safe against concurrent `Register()` calls.

Documentation

- Added a plain-language "note on network access" to the README and the PDF documentation (the page has no login; guidance for shared networks).

1.2.0 — 2026-06-09

First public release.

Added

- `LogWarning(msg, args)` severity level.
- `LogException(ex, context?)` — records an exception with full type, message and stack trace (including inner exceptions).
- `MinLevel` property — severity floor; below-floor messages are dropped at the source.
- `LoggingEnabled` property and page "Logging" button — master record switch, persisted across restarts.

Earlier

- Core library (pre-1.2.0): CWS-hosted single-page log viewer with search, severity filtering, light/dark themes, text export, pause control, bounded ring buffer, disk persistence, millisecond timestamps, incremental polling, and the processor error-log view on hardware appliances.